



Multilingual Documentation of R Packages

Google Summer of Code 2026 – R Project for Statistical Computing

Aditya Bansal

30 March, 2026

Table of contents

1 Contributor Information	3
2 Project Selection	3
3 Synopsis	3
4 Problem Statement and Motivation	4
5 Background and Prior Art	4
5.1 The Rd Macro System	4
5.2 What rhelpi18n Does	5
5.3 Root Causes	6
6 Proposed Solution	7
7 Technical Design	8
7.1 Component Overview	8
7.2 Data Flow	13
8 Implementation Plan	14
8.1 Community Bonding Period (May 1 to May 24, 2026)	14
8.2 Timeline	14
8.2.1 Phase 1: Core Infrastructure (Weeks 1 to 3, May 25 to June 14, ~80 hours) .	14
8.2.2 Phase 2: Translation Store and Substitution (Weeks 4 to 6, June 15 to July 5, ~90 hours)	15
8.2.3 Mid-term Evaluation (July 6 to July 10, 2026)	15
8.2.4 Phase 3: Dynamic Content and UI (Weeks 7 to 9, July 13 to August 2, ~100 hours)	15
8.2.5 Phase 4: Documentation and Release Preparation (Weeks 10 to 12, August 3 to August 24, ~80 hours)	16
8.2.6 Final Evaluation (August 24 to August 31, 2026)	17
9 Deliverables	18
10 Testing Plan	18
11 Risks and Mitigations	19
12 Communication Plan	20
13 Availability and Other Commitments	20
14 Prior Work and Proof of Execution	20

1 Contributor Information

Field	Details
Name	Aditya Bansal
Email	aditya2017rks@gmail.com
GitHub	adit-0132
University	Jaypee Institute of Information Technology, Noida
Degree	B.Tech, Computer Science (Year 3)
Concurrent Degree	B.S. in Data Science and Programming, IIT Madras
Timezone	IST (UTC+5:30)
Availability	40 hours per week

2 Project Selection

Field	Details
Organization	R Project for Statistical Computing
Project Title	Multilingual Documentation of R Packages
Mentors	Heather Turner, Elio Campitelli
Project Size	350 hours
Package Name	To be confirmed with mentors

3 Synopsis

This project implements a new R package that brings multilingual documentation to R help pages with correct support for dynamic content generated by Rd macros. The package intercepts `utils:::getHelpFile()`, applies section-level translations from installable modules, and renders a per-section toggle so users can switch between translated and original text without changing environment variables.

4 Problem Statement and Motivation

R’s documentation is written in English by convention. A small number of packages, such as `labstatR` (Italian) and `chilemapas` (Spanish), are documented in other languages, but this isolates them from users who do not speak those languages. Package authors must choose between accessibility for their target audience and visibility in the wider R ecosystem.

The `rhelphi18n` package is an existing attempt to solve this with installable translation modules. It works for simple, static help pages. However, it fails silently on pages that contain `\Sexpr` macros, which are used in a large fraction of well-maintained packages to generate content at install time or render time.

Before multilingual documentation can be adopted at scale, a solution that handles dynamic content correctly is needed.

5 Background and Prior Art

5.1 The Rd Macro System

R help pages are stored as `.Rd` files and compiled into binary `.rdp` databases when a package is installed. The `\Sexpr` macro allows R expressions to be embedded in documentation and evaluated at one of three stages:

- `stage=build`: evaluated when R CMD build creates the source tarball.
- `stage=install`: evaluated once when the package is installed; the result is baked into the compiled `.rdp` database.
- `stage=render`: evaluated every time the user opens the help page.

This is demonstrated concretely in the `gsocproposal` package I built as the coding test for this project. The following Rd snippet shows both stages:

```
#' Deadline: \Sexpr[results=text,stage=install]{
#'   as.character(gsocproposal:::pkg_env$deadline)
#' }
#'
#' \Sexpr[results=text,stage=render]{
#'   dl      <- gsocproposal:::pkg_env$deadline
```

```

#'   days_left <- as.integer(dl - Sys.Date())
#'   if (days_left > 0) {
#'     paste0("The deadline is in ", days_left, " day(s).")
#'   } else if (days_left == 0) {
#'     "The deadline is today."
#'   } else {
#'     "The deadline has passed, try next year!"
#'   }
#' }

```

The install-stage expression writes a fixed date string into the compiled database. The render-stage expression runs fresh every time `?submit_proposal` is called, producing a different message depending on the current system date.

5.2 What `rhelphi18n` Does

I studied the `rhelphi18n` source code directly to understand exactly where its implementation breaks down. The five relevant files are: `getHelpFile-shim.R`, `module-create.R`, `rd-flatten.R`, `rd-translate.R`, and `rd-unflatten.R`.

The full pipeline is as follows.

Module creation (`module-create.R`, line 42):

```

rd_files <- list.files(
  file.path(package_path, "man"),
  pattern = "*.Rd",
  full.names = TRUE
)
macros <- tools::loadPkgRdMacros(package_path)
i18n_translation_templates(rd_files, ...)

```

It reads directly from the `man/` source directory of the package before installation. No macros have been evaluated at this point. The raw `\Sexpr[stage=install]{...}` expression is what gets written into the YAML `original` field, not its output.

Flattening (`rd-flatten.R`, `to_text()` function):

```

to_text <- function(x) {
  tag <- attr(x, "Rd_tag")
  if (length(x) == 0) return("")
  if (is.character(x) || !is.null(tag) && !startsWith(tag, "\\")) {
    return(x[[1]])
  }
  text <- as.character(setRd(x), deparse = TRUE)
  paste(text, collapse = "")
}

```

Every Rd node, regardless of type, is collapsed to a character string. A live render-stage `\Sexpr` node becomes its raw expression text. All structural information is lost.

Version check (`rd-translate.R`, line 27):

```

version_matches <-
  original[[section]]$original == translation[[section]]$original

```

At help time, the compiled Rd database already contains the evaluated output of install-stage expressions (for example, "2025-03-31"). The YAML `original` contains the unevaluated source string `\Sexpr[stage=install]{as.character(.pkg_env$deadline)}`. These never match. The version check fails and the translation is silently discarded.

Unflattening (`rd-unflatten.R`):

```

rd_unflatten <- function(rd_flat) {
  text <- list2rdtext(rd_flat)
  file <- tempfile(fileext = ".rd")
  writeLines(text, file)
  tools::parse_Rd(file)
}

```

The reconstructed text is written to a temp file and re-parsed. This step is not the root problem: if a translator correctly preserved a `\Sexpr` string, it would be re-parsed as a live node. The problem is that the upstream steps lose the node before it ever reaches here.

5.3 Root Causes

Problem	Location	Root cause
Install-stage Sexpr never translates	<code>module-create.R</code> line 42, <code>rd-translate.R</code> line 27	Module is created from source <code>man/</code> files before install; version check fails because pre- and post-install strings differ
Render-stage Sexpr lost or corrupted	<code>rd-flatten.R</code> <code>to_text()</code>	No placeholder mechanism; node is collapsed to raw expression string with no way to restore it
Argument formatting issue with ...	<code>rd-unflatten.R</code> <code>section2char()</code>	Special Rd characters in argument names are not escaped before writing to the temp file

None of these are minor bugs. They are consequences of a design choice to work at the text level rather than the parse tree level. The new package will not attempt to patch these. It will adopt a different design from the start.

6 Proposed Solution

The new package works at the Rd parse tree level throughout. It never collapses nodes to plain text except for sections that are confirmed to contain no dynamic content. The key design decisions are:

1. Create translation modules from the installed package's compiled Rd database via `tools::Rd_db()`, not from source `man/` files. At this point, install-stage expressions are already evaluated to plain text.
2. Introduce a placeholder mechanism for render-stage `\Sexpr` nodes. During flattening, each render-stage node is replaced by a numbered token `{SEXP_0}`, `{SEXP_1}`, and so on. The original node objects are stored in a side list. Translators translate the surrounding scaffold text and preserve the tokens verbatim. During unflattening, tokens are replaced with the original node expressions before the Rd is re-parsed.

3. Hook into `utils:::.getHelpFile()` at the same interception point `rhelphi18n` uses, because this is where all help viewers (console, RStudio, browser) converge.
4. Inject a translation notice and a per-section HTML toggle into the rendered output, so users can switch between translated and original text without touching environment variables.

7 Technical Design

7.1 Component Overview

The package has six core components.

1. Rd database reader

Reads the compiled Rd database of an installed package and returns a named list of parsed Rd objects, one per help topic.

```
# Uses tools::Rd_db() which returns post-install, post-macro-resolution objects
rd_objects <- tools::Rd_db(package = "glue")
```

This is the correct source for module creation. Install-stage `\Sexpr` nodes have already been evaluated. Their output is present as plain text nodes in the parse tree.

2. Rd flattener with placeholder support

Walks the parse tree of each Rd object and produces a named list of section contents. For each section it classifies nodes as static, install-resolved, or render-stage dynamic.

```
# Pseudocode for the new flatten logic
flatten_section <- function(section_nodes) {
  sexpr_store <- list()
  counter      <- 0L

  result <- lapply(section_nodes, function(node) {
    tag <- attr(node, "Rd_tag")

    # Render-stage Sexpr: replace with placeholder, store original node
    if (identical(tag, "\\Sexpr") &&
        isTRUE(grepl("stage=render", attr(node, "Rd_option")))) {
```



```

    token      <- paste0("{SEXPR_", counter, "}")
    sexpr_store[[as.character(counter)]] <- node
    counter    <- counter + 1L
    return(token)
  }

  # All other nodes: return text content (install-stage already resolved)
  node_to_text(node)
})

list(
  text      = paste(unlist(result), collapse = ""),
  sexpr_store = sexpr_store
)
}

```

The YAML written for the translator looks like this:

```

description:
  original: "The deadline is {SEXPR_0} away."
  sexpr_count: 1
  translation: ~

```

The translator writes:

```

description:
  original: "The deadline is {SEXPR_0} away."
  sexpr_count: 1
  translation: "La fecha limite es {SEXPR_0}."

```

The token `{SEXPR_0}` is treated as opaque. It must appear in the translation exactly as it appears in the original.

3. Translation store and loader

Each translation module is a regular R package with a `translations/` directory of YAML files and a `translations` list object built at load time. The YAML schema extends the `rhelphi18n` format with two new fields: `sexpr_count` and optionally `sexpr_nodes` for documentation purposes. Existing

`rhelapi18n` modules that lack these fields are treated as fully static, which preserves backward compatibility.

```
# Translation module DESCRIPTION fields (same convention as rhelapi18n)
# Package: glue.es
# Suggests: glue (== 1.7.0)
# Language: es

# Loading translations at package attach time
translations <- yaml::read_yaml("translations/as_glue.yaml")
```

4. Substitution layer

Applied at help time after the compiled Rd object is fetched. It walks the parse tree, identifies sections that have a translation, and replaces their content.

For sections without render-stage placeholders, replacement is a direct node swap: the translated string is parsed back to an Rd text node.

For sections with render-stage placeholders, the substitution layer:

- Parses the translated scaffold string.
- Locates each `{SEXPR_N}` token in the parsed result.
- Replaces each token with the original live `\Sexpr` node retrieved from the `sexpr_store`.
- Returns the reconstructed subtree.

Because the live `\Sexpr` nodes are restored before the Rd object reaches the renderer, R evaluates them normally. The user sees translated scaffold text with correctly computed dynamic values embedded.

5. `utils:::.getHelpFile()` shim

The interception point follows the same approach as `rhelapi18n`: the shim wraps the internal function, checks for an installed translation module, applies the substitution layer, and returns the modified Rd object.

```
# Simplified shim structure
shim <- function(file) {
  rd      <- original_getHelpFile(file)
  pkgname <- attr(file, "pkgname")
```

```

language <- Sys.getenv("LANGUAGE", unset = "en")

module <- find_translation_module(pkgname, language)
if (is.null(module)) return(rd)

translations <- load_translations(module, basename(file))
if (is.null(translations)) return(rd)

rd_substitute(rd, translations)
}

# Installed at package attach time, same pattern as rhelpi18n
utils:::.getHelpFile <- shim

```

6. Translation notice and toggle

After substitution, the shim injects a small HTML block at the top of the rendered page. For each translated section, a toggle link is added that swaps the section content between translated and original text via a minimal inline JavaScript snippet. The toggle stores the original content as a data attribute on the section element at render time.

The toggle works in both the R HTML help browser and the RStudio help pane. Plain text output (console) displays translated text without a toggle, which is still an improvement over no translation.

7.2 Data Flow

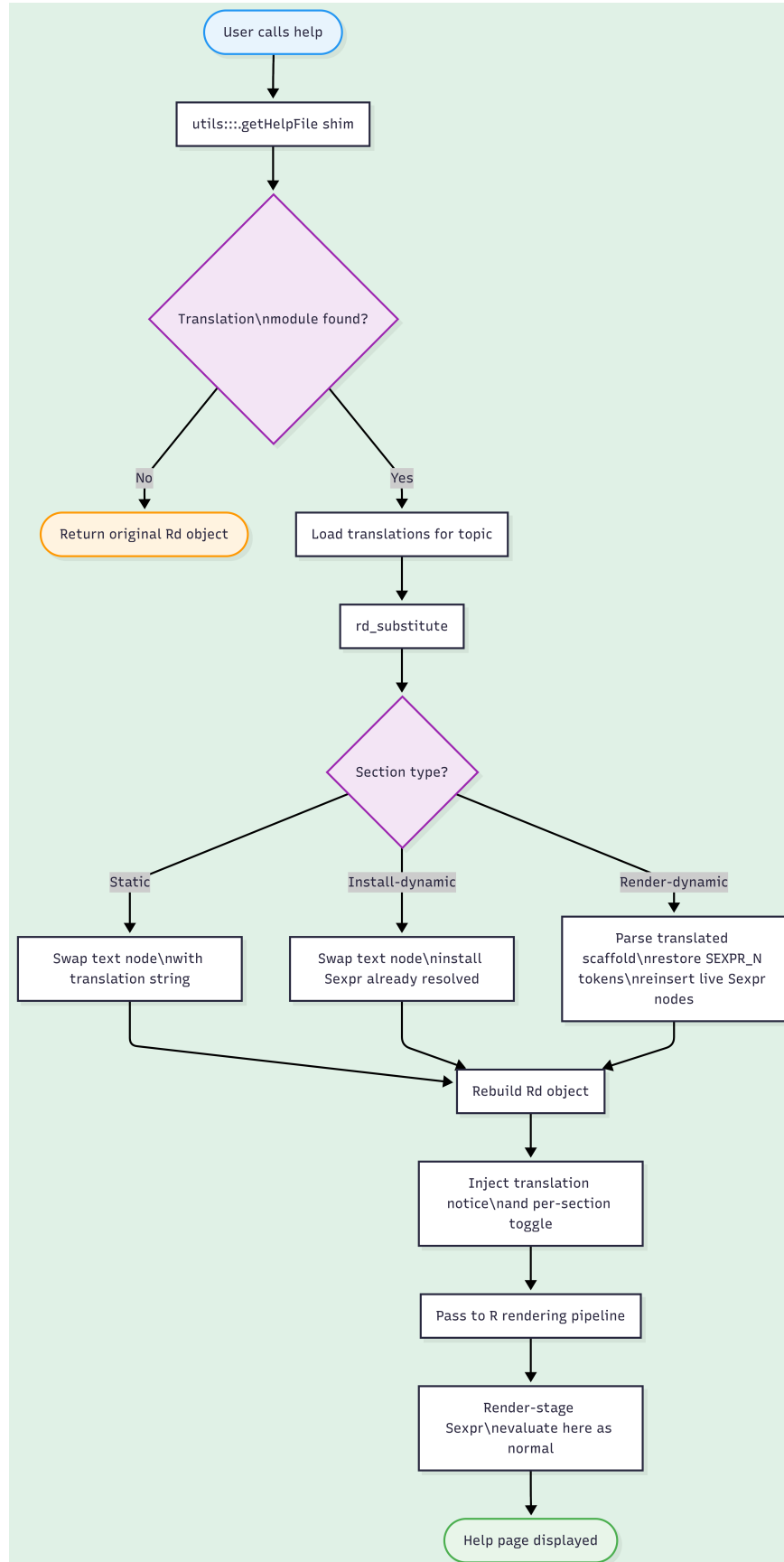


Figure 1: Architecture: data flow from `help()` call through translation pipeline

8 Implementation Plan

8.1 Community Bonding Period (May 1 to May 24, 2026)

- Map the full call chain from `help()` through `utils:::.getHelpFile()` to the HTML and text rendering paths in `tools`.
- Write a prototype that reads a compiled Rd object via `tools::Rd_db()`, modifies one text node, and confirms the modified object renders correctly. This validates the core assumption before any other work begins.
- Agree with mentors on the final package name, YAML schema, and the exact backward compatibility guarantees for existing `rhelphi18n` modules.
- Set up the package skeleton, CI via GitHub Actions, and code style checks with `lintr`.

8.2 Timeline

8.2.1 Phase 1: Core Infrastructure (Weeks 1 to 3, May 25 to June 14, ~80 hours)

Week	Dates	Tasks	Deliverables
1	May 25 to May 31	Implement <code>utils:::.getHelpFile()</code> shim; unit tests for attach and detach	Shim passing tests; unmodified pages returned correctly
2	Jun 1 to Jun 7	Implement Rd flattener with placeholder support; handle all standard section types	Flattener with tests; render-stage nodes replaced by tokens
3	Jun 8 to Jun 14	Implement unflattener with token restoration; round-trip test on pages with and without macros	Round-trip test passing; Rd objects identical before and after no-op translation

8.2.2 Phase 2: Translation Store and Substitution (Weeks 4 to 6, June 15 to July 5, ~90 hours)

Week	Dates	Tasks	Deliverables
4	Jun 15 to Jun 21	Implement <code>tools::Rd_db()</code> based module creator; write YAML with new schema fields	Module creator producing correct YAML from installed packages
5	Jun 22 to Jun 28	Implement substitution layer; handle static and render-dynamic sections; backward compat with rhelpi18n YAML	Substitution layer with tests for all three section types
6	Jun 29 to Jul 5	End-to-end test on a real package with an existing rhelpi18n module; document compatibility gaps	Confirmed backward compatibility; gaps documented

8.2.3 Mid-term Evaluation (July 6 to July 10, 2026)

Expected state: translated sections display correctly for static pages, dynamic content from install-stage macros translates correctly, render-stage scaffold text translates while the computed value remains live, and the original page is returned unchanged when no module is present.

8.2.4 Phase 3: Dynamic Content and UI (Weeks 7 to 9, July 13 to August 2, ~100 hours)

Week	Dates	Tasks	Deliverables
7	Jul 13 to Jul 19	Full test suite against the dummy <code>gsocproposal</code> package; confirm install-stage and render-stage Sexpr both survive translation	Dynamic content test suite passing
8	Jul 20 to Jul 26	Implement translation notice and per-section toggle; test in console, R HTML browser, and RStudio	Toggle working in all three viewers
9	Jul 27 to Aug 2	Edge case handling: custom macros, non-UTF-8 content, non-standard section layouts, partial translations	Edge case tests added and passing

8.2.5 Phase 4: Documentation and Release Preparation (Weeks 10 to 12, August 3 to August 24, ~80 hours)

Week	Dates	Tasks	Deliverables
10	Aug 3 to Aug 9	Write translator vignette; document YAML schema; produce worked example with a popular package	Vignette published; worked example committed

Week	Dates	Tasks	Deliverables
11	Aug 10 to Aug 16	Write developer documentation covering architecture and the interception point; reach 80% test coverage	Developer docs; coverage target met
12	Aug 17 to Aug 24	Final mentor review, bug fixes, README, release preparation	Package ready for public announcement

The scheduled work across all four phases totals approximately 350 hours. Any remaining capacity serves as buffer for refining deliverables or addressing reviewer feedback.

8.2.6 Final Evaluation (August 24 to August 31, 2026)

Criterion	Expected state
Shim intercepts <code>utils:::.getHelpFile()</code> correctly	Complete
Install-stage Sexpr sections translate correctly	Complete
Render-stage Sexpr scaffold text translates; computed value remains live	Complete
Static sections translate with graceful degradation for partial modules	Complete
Translation notice and per-section toggle work in all help viewers	Complete
Backward compatibility with existing <code>rhelpl8n</code> modules confirmed	Complete
Translator vignette and developer documentation published	Complete
Test suite passing with no errors or warnings	Complete

9 Deliverables

ID	Deliverable
D1	Installable R package on GitHub (name to be confirmed with mentors)
D2	<code>utils:::.getHelpFile()</code> shim that intercepts at the post-macro-resolution stage
D3	Rd flattener with placeholder support for render-stage <code>\Sexpr</code> nodes
D4	Translation module creator based on <code>tools::Rd_db()</code> rather than source <code>man/</code> files
D5	Substitution layer handling static, install-dynamic, and render-dynamic sections
D6	Translation notice and per-section toggle for HTML and RStudio viewers
D7	Full test suite against the <code>gsocproposal</code> dummy package with install- and render-stage macros
D8	Backward compatibility with existing <code>rhelpi18n</code> YAML module format
D9	Vignette for translators documenting the YAML schema and workflow
D10	Developer documentation covering architecture, interception point, and maintenance
D11	Worked example with a popular package and a partial translation module (package to be selected with mentor advice)

10 Testing Plan

Unit tests cover the Rd flattener (placeholder insertion for render-stage nodes, correct text extraction for static nodes), the unflattener (token restoration, round-trip identity for pages without translations), the substitution layer (full translation, partial translation, out-of-date translation, no module present), and the module creator (correct YAML schema produced from installed packages).

Integration tests use the `gsocproposal` package as the primary test fixture. This package has both an install-stage `\Sexpr` (the deadline date) and a render-stage `\Sexpr` (the time-remaining message). A hand-authored translation module for it is committed to the repository and used in CI. The integration test confirms that the translated page shows the correct scaffold text in the target language while the computed date value remains live.

A second integration test uses a real `rhelphi18n` module (such as `base.es`) to verify backward compatibility.

All tests use `testthat` edition 3. CI runs on every pull request via GitHub Actions on Linux and macOS across R versions 4.3, 4.4, and 4.5.

11 Risks and Mitigations

Risk	Likelihood	Mitigation
<code>utils:::.getHelpFile()</code> signature changes between R versions	Low	Pin tested R versions in CI; add a startup check that warns if the internal function signature differs from expected
Token restoration fails for nested or multi-line render-stage expressions	Medium	Collect all <code>\Sexpr</code> patterns from CRAN packages during community bonding; add to the test suite before writing the unflattener
Per-section toggle does not render in all help viewers	Medium	Scope the toggle to HTML output only in Phase 3; plain text falls back to translated text with no toggle
Backward compatibility with <code>rhelphi18n</code> modules is incomplete	Medium	Test against all publicly available modules early in Phase 2; document any schema differences clearly

Risk	Likelihood	Mitigation
A popular package uses an undocumented Rd macro type that breaks the flattener	Low	The flattener treats unknown node types as opaque static text, which degrades gracefully rather than erroring

12 Communication Plan

I will open a weekly status issue on the project repository every Monday summarising progress, blockers, and the plan for the coming week. I will attend all scheduled mentor meetings and respond to review comments within one working day. For decisions that affect the public API or the translation module schema, I will post to the relevant R development mailing list or the agreed communication channel for broader input.

13 Availability and Other Commitments

I have no examinations, coursework deadlines, or other commitments during the GSoC coding period. My college (Jaypee Institute of Information Technology) is on vacation from late May through August, which covers the entire coding window. I can contribute 40 hours per week for the full project duration.

14 Prior Work and Proof of Execution

All of my open-source contributions are to R documentation and developer infrastructure, which is the exact domain of this project. I've already become very accustomed to writing Quarto documents, so much so that I've written this proposal itself with .qmd. I'm also comfortable with writing vignettes and working with tools like lychee that are popular in the R ecosystem. I have worked directly with Heather on both the `rdevguide` and `r-dev-env` contributions and am familiar with the review process and expectations.

Merged:

- `r-devel/rdevguide` PR #241: R Developer Guide documentation improvement (github.com/r-devel/rdevguide/pull/241)
- `rstudio/shiny` PR #4342: Shiny documentation improvement (github.com/rstudio/shiny/pull/4342)

Open:

- `r-devel/r-dev-env` PR #292: R Dev Environment documentation (github.com/r-devel/r-dev-env/pull/292)
- `r-devel/rdevguide` PR #258: R Developer Guide documentation (github.com/r-devel/rdevguide/pull/258)

Coding test:

The `gsocproposal` package was built as the application test for this project. It implements a deadline-aware `submit_proposal()` function with both install-stage and render-stage `\Sexpr` macros, full roxygen2 documentation, and a `testthat` suite covering all three test levels (Easy, Medium, Hard). The package is also used as the primary integration test fixture in this proposal.

Repository: github.com/adit-0132/multilingual-docs

Write-Up : [Creation of the test package](#)